Implementing ReAct and Reflexion Agents

Table of Contents

---

1. Overview: Two Different Problems

ReAct and Reflexion are often mentioned in the same breath, but they solve different problems and compose well together rather than being alternatives:

	ReAct	**Reflexion**
Solves	How does an agent interleave reasoning with tool use *within* a single attempt?	How does an agent improve *across* multiple attempts at the same task?
Mechanism	Thought → Action → Observation loop	Actor → Evaluator → Self-Reflection loop, with reflections stored in memory
Memory scope	Within one trajectory	Across trials — persists between attempts
Core insight	Grounding reasoning in real observations reduces hallucination	Verbal feedback (in natural language) can substitute for gradient-based weight updates

A useful mental model: **ReAct governs a single trial; Reflexion governs what happens between trials.** A Reflexion agent's "Actor" is often, itself, a ReAct agent.

---

2. ReAct — Reasoning + Acting

ReAct interleaves reasoning traces with actions, so the model doesn't just plan blindly — it grounds each step in a real observation from the environment before deciding the next step. The loop is:

1. **Thought** — the model reasons in natural language about what to do next.
2. **Action** — the model emits a structured tool call.
3. **Observation** — the harness executes the tool and returns the real result.
4. **Repeat** until the model has enough information to produce a final answer.

This pattern has become the dominant architecture for tool-using agents — industry surveys from early 2026 indicate that over two-thirds of production LLM agent deployments use some form of the ReAct pattern, and it measurably outperforms chain-of-thought prompting alone on multi-step tasks that require external data. [1]

---

3. Implementing ReAct From Scratch

The core structure is deliberately simple: a message history, a tool registry, and a loop that alternates between model calls and tool execution.

```python
```

```python
import anthropic

client = anthropic.Anthropic()

TOOLS = [
    {
        "name": "search_docs",
        "description": "Search internal documents for a query string.",
        "input_schema": {
            "type": "object",
            "properties": {"query": {"type": "string"}},
            "required": ["query"],
        },
    },
]

def search_docs(query: str) -> str:
    # placeholder — wire to your actual retrieval system
    return f"3 documents found matching '{query}'"

TOOL_IMPLS = {"search_docs": search_docs}

def react_agent(task: str, max_steps: int = 8) -> str:
    messages = [{"role": "user", "content": task}]

    for step in range(max_steps):
        response = client.messages.create(
            model="claude-sonnet-4-5",
            max_tokens=1024,
            tools=TOOLS,
            messages=messages,
        )
        messages.append({"role": "assistant", "content": response.content})

        if response.stop_reason != "tool_use":
            # No more actions requested — this is the final answer (the "Though
            return "".join(b.text for b in response.content if b.type == "text"

        # Execute the requested action(s) and feed the observation back
        tool_results = []
        for block in response.content:
            if block.type == "tool_use":
                observation = TOOL_IMPLS[block.name](**block.input)
```

```
                tool_results.append({
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": observation,
                })
        messages.append({"role": "user", "content": tool_results})

    return "Reached max steps without a final answer."
```

Each iteration of this loop *is* one Thought→Action→Observation cycle: the model's text content is the "Thought," the `tool_use` block is the "Action," and the `tool_result` fed back is the "Observation." Nothing more exotic than that is required for a working ReAct agent — the pattern is really just tool-calling with the loop made explicit.

---

4. Implementing ReAct in LangGraph

4.1 The Prebuilt Path

LangGraph ships a factory function that wraps the full loop into a compiled graph — you supply a model and a list of tools, and it handles message history, tool routing, and loop termination automatically:

python

```python
from langgraph.prebuilt import create_react_agent

def check_broker_status(broker_id: str) -> str:
    """Look up whether a broker record is active."""
    return f"Broker {broker_id} is active"

graph = create_react_agent(
    "anthropic:claude-sonnet-4-5",
    tools=[check_broker_status],
    prompt="You are a helpful insurance data assistant.",
)

for chunk in graph.stream(
    {"messages": [{"role": "user", "content": "Is broker BRK-1029 active?"}]},
    stream_mode="updates",
):
    print(chunk)
```

> **Note on API churn:** `create_react_agent` from `langgraph.prebuilt` is being deprecated in favor of `create_agent` from the `langchain` package, which provides an equivalent

> factory with a more flexible middleware system.[2] If you're starting a new project, check current LangChain/LangGraph docs for whichever entry point is current — the underlying graph shape (below) hasn't changed, only which import path is the supported one.

4.2 Building It Manually (What the Prebuilt Function Does Internally)

Understanding the manual version matters because it's what you'll extend when you need custom routing (e.g. rerouting to a reflection step, as covered next):

python

```python
from langgraph.graph import StateGraph, END
from langgraph.graph.message import add_messages
from typing import Annotated, TypedDict
from langchain_core.messages import BaseMessage

class ReActState(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]

def agent_node(state: ReActState) -> ReActState:
    response = model_with_tools.invoke(state["messages"])
    return {"messages": [response]}

def should_continue(state: ReActState) -> str:
    last_message = state["messages"][-1]
    return "tools" if last_message.tool_calls else END

workflow = StateGraph(ReActState)
workflow.add_node("agent", agent_node)
workflow.add_node("tools", tool_node)  # a ToolNode wrapping your tool list
workflow.set_entry_point("agent")
workflow.add_conditional_edges("agent", should_continue, {"tools": "tools", END
workflow.add_edge("tools", "agent")

app = workflow.compile()
```

`add_messages` is a reducer supplied by LangGraph — it appends new messages to the running history by message ID rather than overwriting the whole list on every node execution.[3] The `should_continue` conditional edge is the loop-termination logic: if the last model message contains tool calls, route to the `tools` node; otherwise the trajectory is done.

---

5. Reflexion — Verbal Reinforcement Learning

Reflexion reinforces language agents not by updating model weights, but through linguistic feedback: the agent reflects verbally on task feedback signals and maintains that reflective text in an episodic memory buffer, which conditions future attempts. [4] This is a genuinely different lever than ReAct — it operates *between* trials, not within one.

The framework has three components: [5]

Component	Role
Actor	Generates text and actions based on state observations — this is often a ReAct agent itself
Evaluator	Scores the Actor's trajectory or output against a success signal (unit tests passing, an LLM-judged rubric, ground truth)
Self-Reflection	Generates a natural-language explanation of *why* the attempt failed and what to try differently, written in the first person, then stored in memory for the next trial

The reported gains are substantial: Reflexion agents improve on decision-making tasks by 22 percentage points absolute over 12 iterative learning steps, by 20 points on knowledge-intensive reasoning tasks, and by up to 11 points on Python programming tasks. [6] Critically, the self-reflection step itself is what drives the improvement — an ablation that removed the natural-language explanation step (leaving only pass/fail signals) did not improve over the baseline, showing the verbal explanation is doing real work, not just the retry itself. [4]

---

6. Implementing Reflexion From Scratch

```python
```

```python
from dataclasses import dataclass, field

@dataclass
class ReflexionMemory:
    reflections: list[str] = field(default_factory=list)

    def add(self, reflection: str) -> None:
        self.reflections.append(reflection)

    def as_context(self) -> str:
        if not self.reflections:
            return ""
        joined = "\n".join(f"- {r}" for r in self.reflections)
        return f"Lessons from previous attempts:\n{joined}\n"


def actor(task: str, memory: ReflexionMemory) -> str:
    """Generates an attempt, conditioned on past reflections."""
    prompt = f"{memory.as_context()}\nTask: {task}\nProvide your best attempt."
    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=1024,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.content[0].text


def evaluator(task: str, attempt: str, success_criteria: str) -> tuple[bool, st
    """Scores the attempt. Returns (passed, raw_feedback)."""
    prompt = (
        f"Task: {task}\nAttempt: {attempt}\n"
        f"Success criteria: {success_criteria}\n"
        "Does the attempt satisfy the criteria? Answer PASS or FAIL, then expla
    )
    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=256,
        messages=[{"role": "user", "content": prompt}],
    )
    text = response.content[0].text
    return text.strip().upper().startswith("PASS"), text
```

```python
def self_reflect(task: str, attempt: str, feedback: str) -> str:
    """Generates a first-person verbal reflection on the failure."""
    prompt = (
        f"Task: {task}\nMy attempt: {attempt}\nFeedback: {feedback}\n"
        "Write a short, first-person reflection on what went wrong and what "
        "I should do differently next time."
    )
    response = client.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=256,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.content[0].text


def reflexion_loop(task: str, success_criteria: str, max_trials: int = 5) -> st
    memory = ReflexionMemory()

    for trial in range(max_trials):
        attempt = actor(task, memory)
        passed, feedback = evaluator(task, attempt, success_criteria)

        if passed:
            return attempt

        reflection = self_reflect(task, attempt, feedback)
        memory.add(reflection)

    return attempt  # exhausted trials — return the last attempt regardless
```

This mirrors the reference open-source structure closely: an Actor/Evaluator/Self-Reflection triad, bounded by `max_trials`, with an option most implementations also expose to cap `max_reflections` independently of trial count. [5]

---

7. Implementing Reflexion in LangGraph

Reflexion is naturally a cyclic graph: Actor → Evaluator → (conditional) → Self-Reflection → back to Actor, or → END on success.

```python
```

```python
from langgraph.graph import StateGraph, END
from typing import TypedDict

class ReflexionState(TypedDict):
    task: str
    success_criteria: str
    attempt: str
    reflections: list[str]
    trial: int
    max_trials: int
    passed: bool

def actor_node(state: ReflexionState) -> ReflexionState:
    memory_context = "\n".join(f"- {r}" for r in state["reflections"])
    state["attempt"] = actor(state["task"], ReflexionMemory(state["reflections"
    return state

def evaluator_node(state: ReflexionState) -> ReflexionState:
    passed, feedback = evaluator(state["task"], state["attempt"], state["succes
    state["passed"] = passed
    state["_last_feedback"] = feedback  # scratch field, not part of persisted
    return state

def reflect_node(state: ReflexionState) -> ReflexionState:
    reflection = self_reflect(state["task"], state["attempt"], state["_last_fee
    state["reflections"].append(reflection)
    state["trial"] += 1
    return state

def route_after_eval(state: ReflexionState) -> str:
    if state["passed"]:
        return END
    if state["trial"] >= state["max_trials"]:
        return END
    return "reflect"

workflow = StateGraph(ReflexionState)
workflow.add_node("actor", actor_node)
workflow.add_node("evaluator", evaluator_node)
workflow.add_node("reflect", reflect_node)
workflow.set_entry_point("actor")
workflow.add_edge("actor", "evaluator")
workflow.add_conditional_edges("evaluator", route_after_eval, {"reflect": "refl
```

```
workflow.add_edge("reflect", "actor")

app = workflow.compile()
```

This is functionally the same shape as a `worker → reflection_agent` architecture with a confidence-based reroute: the `evaluator` node is your scoring/confidence step, and the conditional edge deciding whether to loop back through `reflect` is exactly the mechanism a reflection agent uses to send work back for another pass with concrete correction hints attached, rather than a bare "try again."

A fuller version of this pattern — generate an initial response, critically reflect on it, autonomously gather new grounding information (e.g. web search), then revise into a final answer — is a well-documented multi-stage LangGraph workflow combining Pydantic for structured outputs, an LLM for reasoning, and a search tool for grounding. [7]

---

8. Combining ReAct and Reflexion

The two compose directly: use a ReAct loop as the **Actor**, so each trial is itself a full reason-act-observe trajectory, and Reflexion governs whether that whole trajectory gets retried with accumulated verbal feedback.

python

```python
def react_actor(task: str, memory: ReflexionMemory, max_steps: int = 8) -> str:
    context = memory.as_context()
    return react_agent(f"{context}\n{task}", max_steps=max_steps)  # from Secti

def reflexion_with_react_actor(task: str, success_criteria: str, max_trials: in
    memory = ReflexionMemory()
    for trial in range(max_trials):
        attempt = react_actor(task, memory)
        passed, feedback = evaluator(task, attempt, success_criteria)
        if passed:
            return attempt
        memory.add(self_reflect(task, attempt, feedback))
    return attempt
```

In LangGraph terms, this just means the `actor` node in Section 7's graph is itself a compiled ReAct subgraph (from Section 4.2) invoked as a single step, rather than a bare model call — LangGraph supports nesting compiled graphs as nodes for exactly this kind of composition.

---

9. Practical Notes

1. **Bound both loops independently.** `max_steps` (within a ReAct trial) and `max_trials` (across Reflexion attempts) are separate circuit breakers — an unbounded inner loop can blow your budget long before the outer Reflexion loop ever gets a chance to reflect.
2. **The Evaluator's reliability caps the whole system.** Reflexion's gains depend entirely on the Evaluator giving a trustworthy pass/fail signal — an LLM-as-judge Evaluator that isn't calibrated against ground truth will reinforce confident-but-wrong attempts just as readily as correct ones.
3. **Store reflections, not raw failed attempts.** The mechanism that improves performance is the *distilled natural-language lesson*, not the full failed trajectory — dumping entire failed attempts back into context bloats tokens without the benefit self-reflection specifically provides. [4]
4. **Sanitize tool observations before they re-enter context**, especially in ReAct loops that touch external/untrusted data — strip HTML, filter instruction-like patterns, and truncate long outputs to avoid both prompt injection and context flooding. [8]
5. **Prefer parallel tool dispatch when calls are independent.** Modern LangGraph tool nodes support concurrent dispatch with per-call timeouts by default; in a hand-rolled loop, use `asyncio.gather()` for independent tool calls to cut latency. [8]
6. **Don't conflate "more trials" with "better agent."** Reflexion's benefit comes from the quality of the verbal reflection, not sheer retry count — a low `max_trials` with a sharp Evaluator and Self-Reflection step often outperforms a high trial budget with a weak one.

---

10. Summary

Question	Answer
What does ReAct add over plain tool-calling?	Makes the reason→act→observe loop explicit and grounds each reasoning step in a real observation
What does Reflexion add over ReAct alone?	A cross-trial memory of verbal, first-person lessons that condition future attempts, without weight updates
Minimum components for ReAct	Message history + tool registry + loop with a stop condition
Minimum components for Reflexion	Actor + Evaluator + Self-Reflection + a memory buffer for reflections
Where they compose	Reflexion's Actor can itself be a ReAct agent — the two solve different axes of the same problem
LangGraph mapping	ReAct → `agent`/`tools` cyclic graph; Reflexion → `actor`/`evaluator`/`reflect` cyclic graph, optionally nesting the ReAct graph as the actor node

11. Sources

Note: LangGraph's prebuilt agent APIs have churned recently (`create_react_agent` → `create_agent`) — verify current import paths against LangChain's docs before shipping. The Reflexion paper itself (Shinn et al., NeurIPS 2023) remains the authoritative source for the framework's theoretical grounding; the code above is an illustrative reference implementation, not the paper's original codebase.

1. Build a ReAct Agent with LangGraph TypeScript (2026) — LangGraph JS ↩
2. create_react_agent | langgraph.prebuilt — LangChain Reference ↩
3. ReAct agent from scratch with Gemini and LangGraph — Google AI for Developers ↩
4. Reflexion: Language Agents with Verbal Reinforcement Learning — Shinn et al., arXiv ↩ ↩2 ↩3
5. kargarisaac/reflexion — Reflexion Agent implementation based on smolagents — GitHub ↩ ↩2
6. Reflexion: Language Agents with Verbal Reinforcement Learning — Shinn et al., arXiv (HTML version) ↩
7. Building a Self-Correcting AI: A Deep Dive into the Reflexion Agent with LangChain and LangGraph — Medium ↩

8. ReAct Agent Pattern: The Complete Developer Implementation Guide for 2026 — RockB ↩ ↩[2]